

Sprint Documentation

Sprint 14	Start Date: 25/03/2026	Final Date: 30/03/2026
Projetc Name:	Civitas-backend	
Ano: 5º	Análise e Desenvolvimento de Sistemas - AMS	
Team Members		
Wendell Alves Nascimento		
Felipe Pacheco Bianchini		
Gustavo Henrique Moreira Porto		

Sprint Backlog

Task#	Description	Start Date	Final date
#57	Implementação de Autenticação com JWT	25/03/2026	30/03/2026

Task Description

Task #	Description	Assigned To	Status	Estimated Hours	Logged Hours
#57		Wendell Alves Nascimento, Felipe Pacheco Bianchini, Gustavo Henrique Moreira Porto	Completo	15 horas	15 horas

Sprint Description

 Tarefa: Sprint 14 - Implementação de Autenticação com JWT

 Objetivo

Implementar o sistema de autenticação de usuários, permitindo que um usuário válido realize login no sistema e receba um token JWT para autenticação nas requisições da API.

 Critérios de Aceite

- Criar endpoint de autenticação (login).

Validar credenciais do usuário (email e senha).

Verificar usuário no banco de dados.

Gerar token JWT após autenticação válida.

Retornar token JWT na resposta da requisição.

Retornar erro apropriado caso as credenciais sejam inválidas.

Atualizar documentação da API.



Escopo da Rota

AUTENTICAÇÃO

Nessa rota, os endpoints devem ser:

POST /api/auth/login



Observações

- O endpoint deve receber as credenciais do usuário.
- O sistema deve validar as informações antes de gerar o token.
- O token JWT deve conter informações essenciais do usuário (id, nome e tipousuario)
- O token deve ter validade máxima de 60 minutos. Após isso, o usuário deve ser deslogado do sistema.
- O token será utilizado posteriormente para autenticação nas rotas protegidas da API.

Sprint Results

Objetivo da task

Implementar autenticação de usuários via `POST /api/auth/login`, validando credenciais e retornando um token JWT para uso nas próximas rotas protegidas.

O que foi implementado

- Endpoint `POST /api/auth/login`.
- Validação básica de payload de login (`email` e `senha` obrigatórios).
- Busca de usuário por email no repositório.
- Verificação de senha com BCrypt.
- Bloqueio de autenticação para usuário inexistente, senha inválida ou usuário inativo.
- Geração de JWT com validade de 60 minutos.
- Inclusão das claims `sub`, `name` e `tipousuario`.
- Configuração do middleware `JwtBearer` no host.
- Documentação do esquema Bearer no Swagger.
- Atualização do `README.md` e do arquivo `Civitas.WebAPI.http`.

Alterações técnicas

- Foi adicionado `Microsoft.AspNetCore.Authentication.JwtBearer`.
- Foram criados `AuthController`, `AuthService` e `JwtTokenService`.
- Foram criados DTOs dedicados para request/response de login.
- `IUsuarioRepository` / `UsuarioRepository` ganharam busca por email.
- Foi adicionada seção `Jwt` no `appsettings.json`.

Testes adicionados

- Login válido retornando token com claims esperadas.
- Login com senha inválida retornando `401 Unauthorized`.
- Login com usuário inativo retornando `401 Unauthorized`.
- Login com payload inválido retornando `400 BadRequest`.

Verificações executadas

- `DOTNET\_ROLL\_FORWARD=Major dotnet test Civitas.WebAPI/Civitas.WebAPI.sln --filter AuthEndpointsTests`

Assinaturas